Experiment 10: Installing and Exploring Selenium for Automated Testing

Aim: To install Selenium WebDriver and Selenium IDE, and explore browser automation: launching a browser, locating elements with different strategies, interacting with a form, reading results and taking screenshots.

Software required: Python 3.10+, pip, Google Chrome, Selenium 4.x (ChromeDriver is downloaded automatically by Selenium Manager), Selenium IDE browser extension, VS Code.

Theory: Selenium is an open-source suite for automating web browsers, mainly used for functional testing of web applications. Its components are:

Component	Purpose
Selenium WebDriver	Library (Python, Java, JavaScript, C#) that controls a real browser through its driver (ChromeDriver, GeckoDriver)
Selenium IDE	Chrome/Firefox extension that records and plays back user actions without coding
Selenium Grid	Runs tests in parallel on many machines and browsers
Selenium Manager	Built into Selenium 4.6+; finds or downloads the correct browser driver automatically

WebDriver locates elements using **locators**: `By.ID`, `By.NAME`, `By.CLASS_NAME`, `By.TAG_NAME`, `By.CSS_SELECTOR`, `By.XPATH`, `By.LINK_TEXT` and `By.PARTIAL_LINK_TEXT`. Common actions are `get()`, `find_element()`, `send_keys()`, `click()`, `text`, `get_attribute()`, `is_selected()`, `save_screenshot()`, `refresh()`, `back()`, `forward()` and `quit()`.

Procedure:

1. Check Python: `python3 --version` (Windows: `python --version`).
2. Create and activate a virtual environment: ***`python3 -m venv .venv`*** then `source` ***`venv/bin/activate`*** (Windows: `venv\Scripts\activate`).
3. Install Selenium: ***`pip install selenium`***, and verify with ***`python -c "import selenium; print(selenium.__version__)"`***.
4. Inside the project folder `selenium-lab`, create a folder `event-registration` containing the web page under test (`index.html`, `style.css`, `script.js`, given below).
5. Create s***`elenium_intro.py`*** with the program below and run it with `python` ***`selenium_intro.py`***. Chrome opens with the banner "Chrome is being controlled by automated test software", fills the form and closes.
6. **Selenium IDE:** install it from the Chrome Web Store, open it, choose **Record a new test in a new project**, enter the URL of the form, record filling and submitting the form, stop recording, then click **Run current test** and observe the log. Export the test as Python pytest code.

Folder structure:

```
selenium-lab/
├── selenium_intro.py
└── event-registration/
    ├── index.html
    ├── style.css
    └── script.js
```

`event-registration/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TechFest 2026 - Registration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>TechFest 2026</h1>
    <p class="sub">Event Registration Form</p>

    <form id="regForm" novalidate>
      <label for="name">Full Name</label>
      <input type="text" id="name" placeholder="Enter your full name" required>

      <label for="email">Email</label>
      <input type="email" id="email" placeholder="name@example.com" required>

      <label for="phone">Mobile Number</label>
      <input type="tel" id="phone" placeholder="10-digit mobile number" required>

      <label for="college">College</label>
      <input type="text" id="college" placeholder="College name" required>

      <label for="event">Select Event</label>
      <select id="event" required>
        <option value="">-- Choose an event --</option>
        <option>Hackathon</option>
        <option>Paper Presentation</option>
        <option>Coding Contest</option>
        <option>Robotics Workshop</option>
```

```
      </select>

      <label>Gender</label>
      <div class="radio">
        <input type="radio" name="gender" value="Male" id="m"><label for="m">Male</label>
        <input type="radio" name="gender" value="Female" id="f"><label for="f">Female</label>
        <input type="radio" name="gender" value="Other" id="o"><label for="o">Other</label>
      </div>

      <label for="password">Password</label>
      <input type="password" id="password" placeholder="Min. 6 characters" required>

      <div class="check">
        <input type="checkbox" id="terms">
        <label for="terms">I agree to the terms and conditions</label>
      </div>

      <p id="error" class="error"></p>
      <button type="submit">Register</button>
      <button type="reset" class="reset">Clear</button>
    </form>

    <div id="success" class="success"></div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

event-registration/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: Arial, sans-serif;
  background: linear-gradient(135deg, #4b6cb7, #182848);
  min-height: 100vh;
  display: flex; justify-content: center; align-items: center;
}
.container {
  background: #fff; width: 400px; padding: 30px;
  border-radius: 10px; box-shadow: 0 8px 20px rgba(0,0,0,0.3);
}
h1 { text-align: center; color: #182848; }
.sub { text-align: center; color: #666; margin-bottom: 20px; }
label { display: block; margin-top: 12px; font-weight: bold; font-size: 14px; }
input[type=text], input[type=email], input[type=tel],
input[type=password], select {
  width: 100%; padding: 9px; margin-top: 5px;
  border: 1px solid #ccc; border-radius: 5px;
}
.radio label, .check label { display: inline; font-weight: normal; margin-right: 10px; }
.radio, .check { margin-top: 8px; }
button {
  width: 48%; padding: 10px; margin-top: 18px; border: none;
  border-radius: 5px; background: #4b6cb7; color: #fff; cursor: pointer;
}
button.reset { background: #999; }
button:hover { opacity: 0.85; }
.error { color: red; font-size: 13px; margin-top: 10px; }
.success { color: green; margin-top: 15px; font-weight: bold; }
```

event-registration/script.js

```
document.getElementById("regForm").addEventListener("submit", function (e) {
  e.preventDefault();

  const name     = document.getElementById("name").value.trim();
  const email    = document.getElementById("email").value.trim();
  const phone    = document.getElementById("phone").value.trim();
  const college  = document.getElementById("college").value.trim();
  const event    = document.getElementById("event").value;
  const gender   = document.querySelector('input[name="gender"]:checked');
  const password = document.getElementById("password").value;
  const terms    = document.getElementById("terms").checked;
  const error    = document.getElementById("error");
  const success  = document.getElementById("success");

  error.textContent = "";
  success.textContent = "";

  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const phonePattern = /^[6-9][0-9]{9}$/;

  if (name === "" || college === "") {
    error.textContent = "Name and College are required.";
  } else if (!emailPattern.test(email)) {
    error.textContent = "Please enter a valid email address.";
  } else if (!phonePattern.test(phone)) {
    error.textContent = "Mobile number must be 10 digits starting with 6-9.";
  } else if (event === "") {
    error.textContent = "Please select an event.";
  } else if (!gender) {
    error.textContent = "Please select your gender.";
```

```
  } else if (password.length < 6) {
    error.textContent = "Password must be at least 6 characters.";
  } else if (!terms) {
    error.textContent = "You must accept the terms and conditions.";
  } else {
    success.textContent = "Thank you " + name + "! You are registered for " + event + ".";
    document.getElementById("regForm").reset();
  }
});
```

Program: `selenium_intro.py`

```
# Experiment 10: Exploring Selenium WebDriver
import os
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

# 1. Launch Chrome (Selenium Manager finds the matching ChromeDriver automatically)
driver = webdriver.Chrome()
driver.set_window_size(1280, 1050)

# 2. Open the event registration form (page under test)
page = "file://" + os.path.abspath("event-registration/index.html")
driver.get(page)
print("Page title :", driver.title)
print("Current URL:", driver.current_url)

# 3. Locating elements using different locator strategies
heading  = driver.find_element(By.TAG_NAME, "h1")
subtitle = driver.find_element(By.CLASS_NAME, "sub")
name_box = driver.find_element(By.ID, "name")
gender   = driver.find_elements(By.NAME, "gender")
email    = driver.find_element(By.CSS_SELECTOR, "input[type='email']")
register = driver.find_element(By.XPATH, "//button[text()='Register']")
print("By.TAG_NAME     h1           ->", heading.text)
print("By.CLASS_NAME   sub          ->", subtitle.text)
print("By.ID           name         -> placeholder:", name_box.get_attribute("placeholder"))
print("By.NAME         gender       ->", len(gender), "radio buttons")
print("By.CSS_SELECTOR email input  -> placeholder:", email.get_attribute("placeholder"))
print("By.XPATH        Register btn ->", register.text)

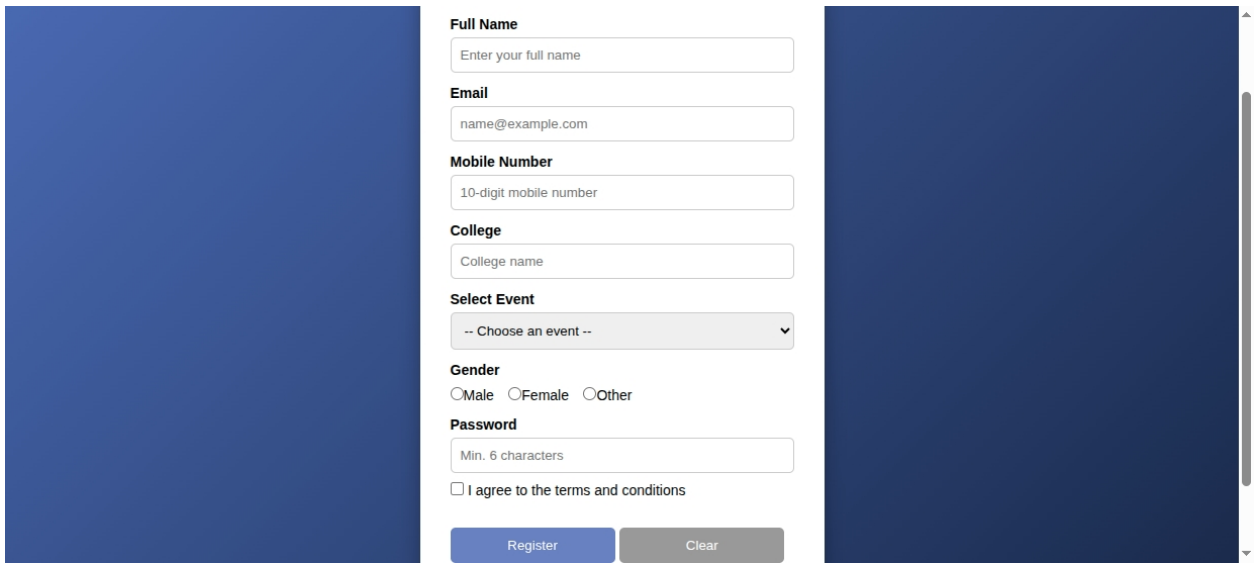
# 4. Interacting with elements
name_box.send_keys("Ravi Kumar")
email.send_keys("ravi@gmail.com")
driver.find_element(By.ID, "phone").send_keys("9876543210")
driver.find_element(By.ID, "college").send_keys("ABC Engineering College")
Select(driver.find_element(By.ID, "event")).select_by_visible_text("Hackathon")
driver.find_element(By.ID, "m").click()
driver.find_element(By.ID, "password").send_keys("ravi@123")
driver.find_element(By.ID, "terms").click()
print("Male selected  :", driver.find_element(By.ID, "m").is_selected())
print("Terms accepted :", driver.find_element(By.ID, "terms").is_selected())
register.click()

# 5. Reading the result and taking a screenshot
message = driver.find_element(By.ID, "success").text
print("Result message :", message)
driver.save_screenshot("registration_result.png")
print("Screenshot saved: registration_result.png")

# 6. Browser navigation commands
driver.refresh()
print("After refresh, Name field empty:", driver.find_element(By.ID, "name").get_attribute("value") == "")
time.sleep(1)
driver.quit()
print("Browser closed.")
```

Output :

```
(.venv) bannu@bannu-Precision-7520:~/Documents/experiment10$ pip install selenium
Collecting selenium
(.venv) bannu@bannu-Precision-7520:~/Documents/experiment10$ python -c "import selenium; print(selenium.__version__
4.50.0
(.venv) bannu@bannu-Precision-7520:~/Documents/experiment10$ python selenium_intro.py
Page title : TechFest 2026 - Registration
Current URL: file:///home/bannu/Documents/experiment10/event-registration/index.html
By.TAG_NAME     h1           -> TechFest 2026
By.CLASS_NAME   sub          -> Event Registration Form
By.ID           name         -> placeholder: Enter your full name
By.NAME         gender       -> 3 radio buttons
By.CSS_SELECTOR email input  -> placeholder: name@example.com
By.XPATH        Register btn -> Register
Male selected  : True
Terms accepted : True
Result message : Thank you Ravi Kumar! You are registered for Hackathon.
Screenshot saved: registration_result.png
After refresh, Name field empty: True
Browser closed.
```



Result: Selenium WebDriver was installed and explored: Chrome was launched and controlled by a Python script, elements were located using six locator strategies, a form was filled and submitted automatically, and the result was verified and saved as a screenshot. Selenium IDE was used to record and play back the same test.

Experiment 11: Simple JavaScript Program Tested using Selenium

Aim: To write a simple calculator program in JavaScript and test its functionality automatically using Selenium WebDriver with the Python `unittest` framework.

Software required: VS Code, Google Chrome, Python 3, Selenium 4 (pip install selenium).

Theory: Automated functional testing checks that an application behaves as expected by driving its user interface the way a user would. Each **test case** sets up a known state, performs actions (enter numbers, click a button) and uses **assertions** to compare the actual output with the expected output. Python's `unittest` module provides the structure: `setUpClass` and `tearDownClass` run once (open and close the browser), `setUp` runs before every test (reload the page), and each method starting with `test_` is one test case. A test passes when all its assertions hold.

Program:



`calculator/index.html`

```
<!DOCTYPE html>                                      <!-- Declares the document as HTML5 -->
<html lang="en">                                     <!-- Root element; page language is English -->
<head>                                               <!-- Head: information about the page, not shown on screen -->
 <meta charset="UTF-8">                             <!-- Character encoding so all symbols (×, ÷) display correctly --
>
 <title>Simple Calculator</title>                   <!-- Text shown on the browser tab; Selenium checks this title →

 <style>                                            /* Internal CSS starts: styles for this page */
 body { font-family: Arial, sans-serif; background: #f0f4f8; display: flex; justify-content: center; padding-top:
60px; }  /* Font, light background, centre the card horizontally, 60px gap at top */
 .calc { background: #fff; padding: 30px; border-radius: 10px; box-shadow: 0 4px 12px rgba(0,0,0,0.15); width:
360px; }  /* White card with padding, rounded corners, shadow, fixed width */
 h2 { text-align: center; color: #182848; margin-top: 0; }  /* Centred dark-blue heading with no top margin */
 input { width: 100%; padding: 10px; margin: 8px 0; box-sizing: border-box; font-size: 16px; border: 1px solid
#ccc; border-radius: 5px; }  /* Full-width number boxes with padding, spacing and a grey border */
 .buttons { display: flex; gap: 8px; margin-top: 8px; }  /* Places the buttons side by side with 8px gaps */
 button { flex: 1; padding: 12px; font-size: 18px; border: none; border-radius: 5px; background: #4b6cb7; color:
#fff; cursor: pointer; }  /* Equal-width blue buttons with white text and a hand cursor */
 #clear { background: #999; }                       /* Clear button is grey to look different */
 #result { margin-top: 18px; font-size: 20px; font-weight: bold; text-align: center; min-height: 24px; }  /* Result
area: large bold centred text; keeps its height even when empty */
 .ok { color: green; }                              /* Class used for a correct result (green) */
 .err { color: red; }                               /* Class used for an error message (red) */
 </style>                                             <!-- End of internal CSS →

</head>                                                <!-- End of head -->
<body>                                                 <!-- Body: visible content of the page -->
 <div class="calc">                                  <!-- Container card for the calculator -->
 <h2>Simple Calculator</h2>                         <!-- Heading shown on the page -->
 <input type="number" id="num1" placeholder="Enter first number">   <!-- First number; id "num1" is used by
JavaScript and Selenium -->
 <input type="number" id="num2" placeholder="Enter second number">  <!-- Second number; id "num2" -->
 <div class="buttons">                                <!-- Row of operator buttons -->
 <button id="add">+</button>                       <!-- Addition button, id "add" -->
 <button id="sub">-</button>                       <!-- Subtraction button, id "sub" -->
 <button id="mul">&times;</button>                 <!-- Multiplication button (× symbol), id "mul" -->
```

```
      <button id="div">&divide;</button>          <!-- Division button (÷ symbol), id "div" -->
    </div>                                        <!-- End of operator row -->
    <div class="buttons"><button id="clear">Clear</button></div>  <!-- Separate row with the Clear button, id
"clear" -->
    <div id="result"></div>                       <!-- Empty area where the answer or error is written by
JavaScript -->
  </div>                                          <!-- End of calculator card -->
  <script src="calc.js"></script>                 <!-- Loads the JavaScript file after the page elements exist -->
</body>                                           <!-- End of body -->
</html>                                           <!-- End of HTML document -->
```

calculator/calc.js

```
// Function that performs the arithmetic; a and b are numbers, op is the operation name
function calculate(a, b, op) {
  switch (op) {                                          // Choose the operation based on op
    case "add": return a + b;                            // "add" -> return the sum
    case "sub": return a - b;                            // "sub" -> return the difference
    case "mul": return a * b;                            // "mul" -> return the product
    case "div":                                          // "div" -> division needs a check first
      if (b === 0) throw new Error("Cannot divide by zero");  // Raise an error if the divisor is 0
      return a / b;                                      // Otherwise return the quotient
  }                                                      // End of switch
}                                                        // End of calculate()

// Function called when an operator button is clicked; op tells which button
function showResult(op) {
  const n1 = document.getElementById("num1").value;      // Read the text typed in the first box
  const n2 = document.getElementById("num2").value;      // Read the text typed in the second box
  const result = document.getElementById("result");      // Get the element where the answer is shown

  if (n1 === "" || n2 === "") {                          // If either box is empty...
    result.className = "err";                            // ...use the red error style
    result.textContent = "Please enter both numbers";    // ...show the error message
    return;                                              // ...and stop here
  }                                                      // End of empty-input check
  try {                                                  // Try the calculation (it may throw an error)
    const value = calculate(parseFloat(n1), parseFloat(n2), op);  // Convert text to numbers and calculate
    result.className = "ok";                             // Use the green success style
    result.textContent = "Result: " + value;             // Display the answer
  } catch (e) {                                          // If calculate() threw an error (divide by zero)...
    result.className = "err";                            // ...use the red error style
    result.textContent = "Error: " + e.message;          // ...display the error message
  }                                                      // End of try...catch
}                                                        // End of showResult()

["add", "sub", "mul", "div"].forEach(op => {             // Loop over the ids of the four operator buttons
  document.getElementById(op).addEventListener("click", () => showResult(op));  // On click, call showResult with
that operation
});                                                      // End of loop

document.getElementById("clear").addEventListener("click", () => {  // When the Clear button is clicked...
  document.getElementById("num1").value = "";            // ...empty the first box
  document.getElementById("num2").value = "";            // ...empty the second box
  document.getElementById("result").textContent = "";    // ...remove the result text
});                                                      // End of Clear handler
```

test_calculator.py

```
# Experiment 11: Testing a JavaScript calculator using Selenium + unittest
import os                                        # os module: used to build the full path of the HTML file
import unittest                                  # unittest: Python's built-in testing framework
from selenium import webdriver                   # webdriver: controls the Chrome browser
from selenium.webdriver.common.by import By      # By: locator types (ID, NAME, XPATH, ...)


class CalculatorTest(unittest.TestCase):         # Test class; inheriting TestCase makes each test_ method a test
case

    @classmethod                                 # Decorator: the next method belongs to the class, not one
object
    def setUpClass(cls):                         # Runs ONCE before all tests
        cls.driver = webdriver.Chrome()          # Start Chrome and keep the driver for all tests
        cls.driver.set_window_size(1280, 1050)   # Set the browser window size (width, height in pixels)
        cls.url = "file://" + os.path.abspath("calculator/index.html")  # Full file URL of the calculator page

    def setUp(self):                             # Runs BEFORE every test
        self.driver.get(self.url)                # Reload the page so each test starts with empty boxes

    def calc(self, a, b, op):                    # Helper used by many tests: performs one calculation
        """Enter two numbers, click an operator button and return the result text."""
        d = self.driver                          # Short name for the driver
        d.find_element(By.ID, "num1").send_keys(str(a))  # Find the first box by id and type number a
        d.find_element(By.ID, "num2").send_keys(str(b))  # Find the second box by id and type number b
        d.find_element(By.ID, op).click()        # Click the operator button whose id is op (add/sub/mul/div)
        return d.find_element(By.ID, "result").text  # Read and return the text shown in the result area

    def test_1_page_title(self):                 # Test 1: page title
```

```
        self.assertEqual(self.driver.title, "Simple Calculator")  # Pass if the tab title matches exactly

    def test_2_addition(self):                      # Test 2: addition
        self.assertEqual(self.calc(12, 8, "add"), "Result: 20")  # 12 + 8 must show "Result: 20"

    def test_3_subtraction(self):                   # Test 3: subtraction
        self.assertEqual(self.calc(20, 5, "sub"), "Result: 15")  # 20 - 5 must show "Result: 15"

    def test_4_multiplication(self):                # Test 4: multiplication
        self.assertEqual(self.calc(6, 7, "mul"), "Result: 42")   # 6 x 7 must show "Result: 42"

    def test_5_division(self):                      # Test 5: division
        self.assertEqual(self.calc(15, 4, "div"), "Result: 3.75")  # 15 / 4 must show "Result: 3.75"

    def test_6_divide_by_zero(self):                # Test 6: negative test - divide by zero
        self.assertEqual(self.calc(10, 0, "div"), "Error: Cannot divide by zero")  # Must show the error message

    def test_7_empty_input(self):                   # Test 7: negative test - no numbers entered
        self.driver.find_element(By.ID, "add").click()  # Click + without typing anything
        self.assertEqual(self.driver.find_element(By.ID, "result").text,
                         "Please enter both numbers")  # Must ask for both numbers

    def test_8_clear_button(self):                  # Test 8: Clear button
        self.calc(3, 4, "add")                      # First do a calculation so there is something to clear
        self.driver.find_element(By.ID, "clear").click()  # Click Clear
        self.assertEqual(self.driver.find_element(By.ID, "num1").get_attribute("value"), "")  # First box must be
empty
        self.assertEqual(self.driver.find_element(By.ID, "result").text, "")  # Result area must be empty

    @classmethod                                    # Class-level method again
    def tearDownClass(cls):                         # Runs ONCE after all tests
        cls.driver.quit()                           # Close the browser and end the WebDriver session


if __name__ == "__main__":                          # True only when this file is run directly (python
test_calculator.py)
    unittest.main(verbosity=2)                      # Run all tests; verbosity=2 prints each test name and result
```

Test cases:

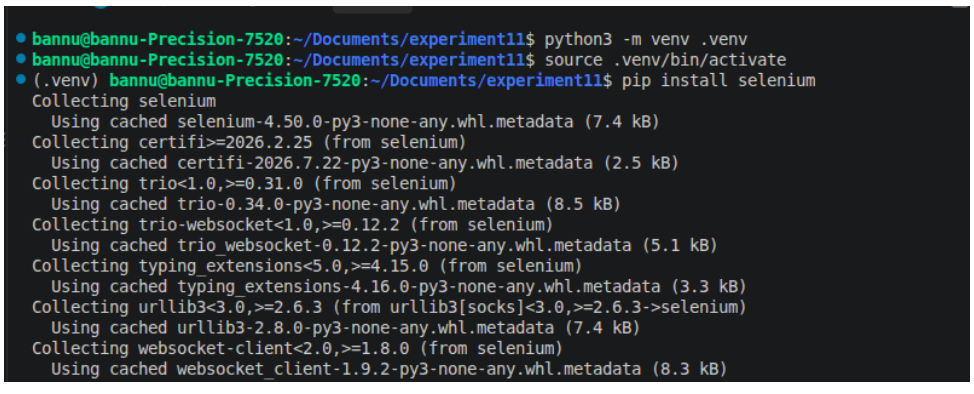
TC	Description	Input	Expected output	Status
1	Page title	Open page	Simple Calculator	Pass
2	Addition	12, 8, +	Result: 20	Pass
3	Subtraction	20, 5, −	Result: 15	Pass
4	Multiplication	6, 7, ×	Result: 42	Pass
5	Division	15, 4, ÷	Result: 3.75	Pass
6	Divide by zero	10, 0, ÷	Error: Cannot divide by zero	Pass
7	Empty input	(none), +	Please enter both numbers	Pass
8	Clear button	3, 4, +, Clear	Inputs and result emptied	Pass

Procedure:

1. Inside `selenium-lab`, create the `calculator` folder with `index.html` and `calc.js`.
2. Open `calculator/index.html` in Chrome and try a few calculations manually.
3. Create `test_calculator.py` in `selenium-lab`.
4. Activate the virtual environment and run `python test_calculator.py`.
5. Observe Chrome opening and running every test; note the summary in the terminal.
6. To see a failing test, change the expected value in `test_2_addition` to `"Result: 21"`, rerun, note the `FAIL` and `AssertionError`, then restore it.

Output (screenshots):

// paste calculator page output which you have captured in lab



```
(.venv) bannu@bannu-Precision-7520:~/Documents/experiment11$ python test_calculator.py
test_1_page_title (__main__.CalculatorTest.test_1_page_title) ... ok
test_2_addition (__main__.CalculatorTest.test_2_addition) ... ok
test_3_subtraction (__main__.CalculatorTest.test_3_subtraction) ... ok
test_4_multiplication (__main__.CalculatorTest.test_4_multiplication) ... ok
test_5_division (__main__.CalculatorTest.test_5_division) ... ok
test_6_divide_by_zero (__main__.CalculatorTest.test_6_divide_by_zero) ... ok
test_7_empty_input (__main__.CalculatorTest.test_7_empty_input) ... ok
test_8_clear_button (__main__.CalculatorTest.test_8_clear_button) ... ok

----------------------------------------------------------------------
Ran 8 tests in 8.390s

OK
```

Result: A simple calculator program was written in JavaScript and tested with Selenium WebDriver and unittest; all 8 automated test cases, covering the four operations, divide by zero, empty input and the Clear button, passed.

Experiment 12: Selenium Test Cases for the Containerized Application

Aim: To develop and execute automated Selenium test cases for a containerized Node.js event registration application, and generate an HTML test report.

Software required: Docker Engine / Docker Desktop, Python 3, Selenium 4, pytest, pytest-html, Google Chrome, VS Code; Minikube and kubectl (optional).

Theory: Once an application runs in a container, the same test suite can verify it in any environment – on a laptop, inside Docker, or behind a Kubernetes Service – just by changing the target URL. **pytest** discovers functions named `test_*` and reports each as PASSED or FAILED. **Fixtures** prepare what tests need: here a module-scoped fixture opens one browser for all tests, and a `page` fixture reloads the app before each test. Because the app updates the page asynchronously (it calls the REST API), tests use **explicit waits** (`WebDriverWait`) instead of fixed sleeps. **pytest-html** turns the results into a shareable HTML report.

Test cases:

TC ID	Test scenario	Test steps / input	Expected result	Actual result	Status
TC01	Page loads	Open BASE_URL	Title "TechFest 2026 - Registration", heading "TechFest 2026"	As expected	Pass
TC02	Form fields present	Check name, email, phone, college, event, Register	All elements displayed	As expected	Pass
TC03	Health endpoint	Open BASE_URL/health	Response contains "status":"UP"	As expected	Pass
TC04	Empty form	Click Register with no input	"Name and College are required."	As expected	Pass
TC05	Invalid email	Email = ravi.gmail.com	"Please enter a valid email address."	As expected	Pass
TC06	Invalid mobile	Mobile = 12345	"Mobile number must be 10 digits starting with 6-9."	As expected	Pass
TC07	No event selected	All fields valid, no event	"Please select an event."	As expected	Pass
TC08	Valid registration	All fields valid, event = Hackathon	Success message; count +1; new row in table	As expected	Pass
TC09	Duplicate email	Register same email again	"This email is already registered." (HTTP 409)	As expected	Pass
TC10	Clear button	Type a name,	Name field	As expected	Pass

TC ID	Test scenario	Test steps / input	Expected result	Actual result	Status
		click Clear	emptied		

Project structure:

```
event-app/                  (application under test)
├── Dockerfile
├── .dockerignore
├── package.json
├── server.js
└── public/
    ├── index.html
    ├── style.css
    └── script.js
selenium-lab/               (test project)
└── test_event_app.py
```

Application under test (source code):

`package.json`

```
{
  "name": "event-registration-app",
  "version": "1.0.0",
  "description": "TechFest 2026 event registration app (containerized with Docker)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.21.0"
  }
}
```

JSON files cannot contain comments, so each line of `package.json` is explained here:

Line	Meaning
`"name"`	Name of the project
`"version"`	Version number of the app
`"description"`	Short description of the app
`"main"`	Entry file that Node.js runs (`server.js`)
`"scripts" → "start"`	Command run by `npm start`: `node server.js`
`"dependencies" → "express"`	Library the app needs; `^4.21.0` means version 4.21.0 or any newer 4.x

`server.js`

```
const express = require("express");                 // Import Express, the web framework that handles HTTP
requests
const path = require("path");                       // Import path module to build file paths safely
const fs = require("fs");                           // Import fs (file system) module to check if a file exists
const os = require("os");                           // Import os module to read the hostname (container ID in
Docker)

const app = express();                              // Create the Express application object
const PORT = process.env.PORT || 3000;              // Port number: use the PORT environment variable, else 3000
const PUBLIC_DIR = path.join(__dirname, "public");  // Absolute path of the "public" folder holding the web files
const registrations = [];                           // In-memory array that stores registered participants

// Startup check: warn if the web page is missing (avoids "Cannot GET /")
if (!fs.existsSync(path.join(PUBLIC_DIR, "index.html"))) {          // If public/index.html does not exist...
 console.error(`ERROR: ${path.join(PUBLIC_DIR, "index.html")} not found.`);  // ...print which file is missing
 console.error("Put index.html, style.css and script.js inside the 'public' folder and rebuild the
image.");  // ...and how to fix it
}                                                   // End of startup check

app.use(express.json());                            // Middleware: convert JSON request bodies into JavaScript
objects
app.use(express.static(PUBLIC_DIR));                // Middleware: serve index.html, style.css, script.js from
public/

// Home page
app.get("/", (req, res) => {                        // When the browser requests "/" (home page)...
 res.sendFile(path.join(PUBLIC_DIR, "index.html")); // ...send back index.html
});                                                 // End of home route

// Health check - used by Docker and Kubernetes
app.get("/health", (req, res) => {                  // When "/health" is requested...
 res.json({ status: "UP", host: os.hostname() });   // ...reply with status UP and the host (container) name
});                                                 // End of health route
```

```
// List all registrations
app.get("/api/registrations", (req, res) => {        // When the list of registrations is requested...
  res.json(registrations);                           // ...send the whole array as JSON
});                                                  // End of list route

// Register a participant
app.post("/api/register", (req, res) => {            // When the form is submitted with POST...
  const { name, email, phone, college, event } = req.body;  // Take the five fields out of the request body
  if (!name || !email || !phone || !college || !event) {    // If any field is missing...
    return res.status(400).json({ error: "All fields are required." });  // ...reply 400 Bad Request with a message
  }                                                  // End of missing-field check
  if (registrations.some(r => r.email === email)) {  // If this email is already in the array...
    return res.status(409).json({ error: "This email is already registered." });  // ...reply 409 Conflict
  }                                                  // End of duplicate check
  const entry = { id: registrations.length + 1, name, email, college, event };  // Build a new record with the next
id
  registrations.push(entry);                         // Add the record to the array
  res.status(201).json({                             // Reply 201 Created with a JSON body containing:
    message: `Thank you ${name}! You are registered for ${event}.`,  // the success message shown on the page,
    id: entry.id,                                    // the new registration id,
    servedBy: os.hostname()                          // and the name of the container that handled the request
  });                                                // End of reply
});                                                  // End of register route

app.listen(PORT, () => {                             // Start the server on PORT; when ready...
  console.log(`Event Registration App running on port ${PORT}`);  // ...print this line (visible in docker logs)
});                                                  // End of listen
```

public/index.html

```
<!DOCTYPE html>                                       <!-- Declares the document as HTML5 -->
<html lang="en">                                      <!-- Root element; page language is English -->
<head>                                                <!-- Head: page information, not displayed -->
 <meta charset="UTF-8">                              <!-- UTF-8 character encoding -->
 <meta name="viewport" content="width=device-width, initial-scale=1.0">  <!-- Makes the page fit mobile screens -->
 <title>TechFest 2026 - Registration</title>         <!-- Browser tab title; checked by test TC01 -->
 <link rel="stylesheet" href="style.css">            <!-- Links the external CSS file -->
</head>                                               <!-- End of head -->
<body>                                                <!-- Visible content starts -->
 <div class="container">                             <!-- White card that holds everything -->
 <h1>TechFest 2026</h1>                             <!-- Main heading; checked by test TC01 -->
 <p class="sub">Event Registration Form</p>         <!-- Sub-heading -->
 <form id="regForm" novalidate>                     <!-- The form; novalidate turns off browser checks so JavaScript
validates -->
 <label for="name">Full Name</label>               <!-- Label for the name box -->
 <input type="text" id="name" placeholder="Enter your full name">       <!-- Name input, id "name" -->
 <label for="email">Email</label>                  <!-- Label for the email box -->
 <input type="email" id="email" placeholder="name@example.com">         <!-- Email input, id "email" -->
 <label for="phone">Mobile Number</label>          <!-- Label for the mobile box -->
 <input type="tel" id="phone" placeholder="10-digit mobile number">     <!-- Mobile input, id "phone" -->
 <label for="college">College</label>              <!-- Label for the college box -->
 <input type="text" id="college" placeholder="College name">            <!-- College input, id "college" -->
 <label for="event">Select Event</label>           <!-- Label for the drop-down -->
 <select id="event">                               <!-- Drop-down list of events, id "event" -->
 <option value="">-- Choose an event --</option>  <!-- Default option with empty value (means nothing chosen) -->
 <option>Hackathon</option>                       <!-- Event option 1 -->
 <option>Paper Presentation</option>              <!-- Event option 2 -->
 <option>Coding Contest</option>                  <!-- Event option 3 -->
 <option>Robotics Workshop</option>               <!-- Event option 4 -->
 </select>                                          <!-- End of drop-down -->
 <p id="error" class="error"></p>                   <!-- Empty paragraph where error messages appear (red) -->
 <button type="submit" id="registerBtn">Register</button>  <!-- Submit button, id "registerBtn" -->
 <button type="reset" class="reset">Clear</button>         <!-- Reset button: clears all inputs; tested by TC10 -->
 </form>                                           <!-- End of form -->
 <div id="success" class="success"></div>          <!-- Empty area where the success message appears (green) -->

 <h3>Registered Participants (<span id="count">0</span>)</h3>  <!-- Heading with live count, id "count" -->
 <table id="regTable">                             <!-- Table listing registrations, id "regTable" -->
 <thead><tr><th>#</th><th>Name</th><th>Event</th></tr></thead>  <!-- Header row: number, name, event -->
 <tbody></tbody>                                  <!-- Body rows are added by script.js -->
 </table>                                          <!-- End of table -->
 <p class="host">Served by: <span id="host">-</span></p>  <!-- Shows which container/pod served the page -->
 </div>                                              <!-- End of card -->
 <script src="script.js"></script>                   <!-- Loads the JavaScript after the elements exist -->
</body>                                               <!-- End of body -->
</html>                                               <!-- End of document -->
```

public/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }  /* All elements: include padding in width; remove default
spacing */
body {                                               /* Styles for the whole page */
 font-family: Arial, sans-serif;                    /* Use Arial font (fallback: any sans-serif font) */
 background: linear-gradient(135deg, #4b6cb7, #182848);  /* Diagonal blue gradient background */
 min-height: 100vh;                                 /* Page is at least the full screen height */
 display: flex; justify-content: center; align-items: center;  /* Flexbox: centre the card horizontally and
vertically */
}                                                    /* End of body styles */
```

```
.container {                                          /* The white card */
  background: #fff; width: 400px; padding: 30px;  /* White background, 400px wide, 30px inner space */
  border-radius: 10px; box-shadow: 0 8px 20px rgba(0,0,0,0.3);  /* Rounded corners and a soft shadow */
}                                                     /* End of card styles */
h1 { text-align: center; color: #182848; }      /* Main heading: centred, dark blue */
.sub { text-align: center; color: #666; margin-bottom: 20px; }  /* Sub-heading: centred, grey, space below */
label { display: block; margin-top: 12px; font-weight: bold; font-size: 14px; }  /* Labels on their own line, bold
*/
input[type=text], input[type=email], input[type=tel], select {  /* Text, email, phone boxes and the drop-down */
  width: 100%; padding: 9px; margin-top: 5px;   /* Full width with inner padding and a small gap above */
  border: 1px solid #ccc; border-radius: 5px;   /* Thin grey border with rounded corners */
}                                                     /* End of input styles */
button {                                              /* Register and Clear buttons */
  width: 48%; padding: 10px; margin-top: 18px; border: none;  /* Two buttons fit side by side; no border */
  border-radius: 5px; background: #4b6cb7; color: #fff; cursor: pointer;  /* Rounded blue buttons, white text, hand
cursor */
}                                                     /* End of button styles */
button.reset { background: #999; }             /* Clear button is grey */
button:hover { opacity: 0.85; }                /* Slightly fade a button when the mouse is over it */
.error { color: red; font-size: 13px; margin-top: 10px; }  /* Error messages in small red text */
.success { color: green; margin-top: 15px; font-weight: bold; }  /* Success message in bold green */
h3 { margin-top: 22px; font-size: 16px; color: #182848; }  /* "Registered Participants" heading */
table { width: 100%; border-collapse: collapse; margin-top: 8px; font-size: 13px; }  /* Full-width table, single
borders */
th, td { border: 1px solid #ddd; padding: 6px; text-align: left; }  /* Cell borders, padding, left-aligned text */
th { background: #eef1f8; }                        /* Light blue header cells */
.host { margin-top: 12px; font-size: 12px; color: #888; text-align: right; }  /* Small grey "Served by" text at the
right */
```

public/script.js

```
async function loadRegistrations() {                        // Function that fetches and displays all registrations
 const res = await fetch("/api/registrations");             // Send GET request to the server and wait for the reply
 const data = await res.json();                             // Convert the JSON reply into a JavaScript array
 const tbody = document.querySelector("#regTable tbody");  // Find the table body where rows go
 tbody.innerHTML = "";                                      // Remove old rows before adding fresh ones
 data.forEach(r => {                                        // For every registration record r...
 tbody.innerHTML += `<tr><td>${r.id}</td><td>${r.name}</td><td>${r.event}</td></tr>`;  // ...add a row with id,
name, event
 });                                                        // End of loop
 document.getElementById("count").textContent = data.length;  // Show the total number of registrations
}                                                            // End of loadRegistrations()

async function loadHost() {                                  // Function that shows which server/container answered
 const res = await fetch("/health");                        // Call the health endpoint
 const data = await res.json();                             // Read the JSON reply
 document.getElementById("host").textContent = data.host;  // Display the host name after "Served by:"
}                                                            // End of loadHost()

document.getElementById("regForm").addEventListener("submit", async function (e) {  // Run this when the form is
submitted
 e.preventDefault();                                        // Stop the browser from reloading the page
 const name    = document.getElementById("name").value.trim();     // Read name, removing extra spaces
 const email   = document.getElementById("email").value.trim();    // Read email
 const phone   = document.getElementById("phone").value.trim();    // Read mobile number
 const college = document.getElementById("college").value.trim();  // Read college
 const event   = document.getElementById("event").value;           // Read the selected event ("" if none)
 const error   = document.getElementById("error");                 // Element for error messages
 const success = document.getElementById("success");               // Element for the success message
 error.textContent = "";                                    // Clear any old error
 success.textContent = "";                                  // Clear any old success message

 if (!name || !college) { error.textContent = "Name and College are required."; return; }  // Name and college must
not be empty
 if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) { error.textContent = "Please enter a valid email address.";
return; }  // Email must look like x@y.z
 if (!/^[6-9][0-9]{9}$/.test(phone)) { error.textContent = "Mobile number must be 10 digits starting with 6-9.";
return; }  // 10 digits, first digit 6-9
 if (!event) { error.textContent = "Please select an event."; return; }  // An event must be chosen

 const res = await fetch("/api/register", {                   // Send the data to the server...
 method: "POST",                                            // ...using the POST method
 headers: { "Content-Type": "application/json" },            // ...telling the server the body is JSON
 body: JSON.stringify({ name, email, phone, college, event })  // ...with the five fields converted to JSON text
 });                                                          // Wait for the server's reply
 const data = await res.json();                               // Read the JSON reply
 if (res.ok) {                                                // If the status is 2xx (success)...
 success.textContent = data.message;                        // ...show the thank-you message
 this.reset();                                              // ...clear the form
 loadRegistrations();                                       // ...refresh the table and count
 } else {                                                     // Otherwise (400 or 409)...
 error.textContent = data.error;                            // ...show the server's error message
 }                                                            // End of if/else
});                                                           // End of submit handler

loadRegistrations();                                          // Fill the table as soon as the page opens
loadHost();                                                   // Show the serving host as soon as the page opens
```

`Dockerfile`

```
# Base image: Node.js 20 on the small Alpine Linux
FROM node:20-alpine
# Working directory inside the container; later commands run here
WORKDIR /app
# Copy package.json (and package-lock.json if present) first, for faster rebuilds (layer caching)
COPY package*.json ./
# Install only the libraries needed to run the app (Express), not development tools
RUN npm install --omit=dev
# Copy the rest of the project (server.js and the public folder) into /app
COPY . .
# Document that the app listens on port 3000
EXPOSE 3000
# Every 30 s Docker calls /health; if it fails within 3 s the container is marked unhealthy
HEALTHCHECK --interval=30s --timeout=3s \
 CMD wget -qO- http://localhost:3000/health || exit 1
# Command run when the container starts: "npm start" -> "node server.js"
CMD ["npm", "start"]
```

In a Dockerfile a comment must be on its own line starting with `#`, so each comment is placed above the instruction it explains.

`.dockerignore`

```
# Do not copy local libraries; the image installs its own with npm install
node_modules
# Do not copy npm error logs
npm-debug.log
# Do not copy the Git history
.git
```

`test_event_app.py`

```
import os                                                    # os: read the BASE_URL environment variable
import time                                                  # time: current time, used to make unique emails
import pytest                                                # pytest: test framework (fixtures, test discovery)
from selenium import webdriver                               # webdriver: controls the Chrome browser
from selenium.webdriver.common.by import By                  # By: locator types (ID, TAG_NAME, CSS_SELECTOR, ...)
from selenium.webdriver.support.ui import Select, WebDriverWait  # Select: drop-downs; WebDriverWait: explicit
waits
from selenium.webdriver.support import expected_conditions as EC  # EC: ready-made wait conditions

BASE_URL = os.getenv("BASE_URL", "http://localhost:3000")  # Address of the app; default is the Docker container
UNIQUE = str(int(time.time()))                               # Seconds since 1970 as text, so emails differ in every
run


@pytest.fixture(scope="module")                              # Fixture shared by all tests in this file (one
browser)
def driver():                                                # Fixture name; tests receive it by asking for "driver"
 drv = webdriver.Chrome()                                   # Start Chrome
 drv.set_window_size(1280, 1050)                            # Set the window size in pixels
 yield drv                                                  # Give the browser to the tests; pause here until they
finish
 drv.quit()                                                 # After all tests: close the browser


@pytest.fixture                                              # Fixture run before EACH test that asks for "page"
def page(driver):                                            # Uses the driver fixture above
 driver.get(BASE_URL)                                       # Open the application home page
 WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "regForm")))  # Wait up to 10 s for the
form
 return driver                                               # Give the ready page to the test


def fill_form(d, name="", email="", phone="", college="", event=None):  # Helper: fill the form and click Register
 d.find_element(By.ID, "name").send_keys(name)              # Type the name
 d.find_element(By.ID, "email").send_keys(email)            # Type the email
 d.find_element(By.ID, "phone").send_keys(phone)            # Type the mobile number
 d.find_element(By.ID, "college").send_keys(college)        # Type the college
 if event:                                                   # Only if an event name was given...
 Select(d.find_element(By.ID, "event")).select_by_visible_text(event)  # ...choose it in the drop-down
 d.find_element(By.ID, "registerBtn").click()               # Click the Register button


def text_of(d, element_id):                                  # Helper: wait for a message and return its text
 WebDriverWait(d, 10).until(lambda drv: drv.find_element(By.ID, element_id).text != "")  # Wait until the element
shows text
 return d.find_element(By.ID, element_id).text              # Return that text


def test_tc01_page_loads(page):                              # TC01: page opens correctly
 assert page.title == "TechFest 2026 - Registration"     # Tab title must match
 assert page.find_element(By.TAG_NAME, "h1").text == "TechFest 2026"  # Main heading must match


def test_tc02_form_fields_present(page):                     # TC02: all form controls are visible
 for field in ["name", "email", "phone", "college", "event", "registerBtn"]:  # Loop over the element ids
```

```
        assert page.find_element(By.ID, field).is_displayed()  # Each element must be displayed


def test_tc03_health_endpoint(driver):                      # TC03: health API works
    driver.get(BASE_URL + "/health")                        # Open the /health URL directly
    assert '"status":"UP"' in driver.find_element(By.TAG_NAME, "body").text  # Page must contain "status":"UP"


def test_tc04_empty_form_rejected(page):                    # TC04: empty form is rejected
    page.find_element(By.ID, "registerBtn").click()         # Click Register with nothing filled
    assert text_of(page, "error") == "Name and College are required."  # Expected error message


def test_tc05_invalid_email(page):                          # TC05: wrong email format
    fill_form(page, "Ravi Kumar", "ravi.gmail.com", "9876543210", "ABC Engineering College", "Hackathon")  # Email
has no @
    assert text_of(page, "error") == "Please enter a valid email address."  # Expected error message


def test_tc06_invalid_phone(page):                          # TC06: wrong mobile number
    fill_form(page, "Ravi Kumar", "ravi@gmail.com", "12345", "ABC Engineering College", "Hackathon")  # Only 5
digits
    assert text_of(page, "error") == "Mobile number must be 10 digits starting with 6-9."  # Expected error message


def test_tc07_event_not_selected(page):                     # TC07: no event chosen
    fill_form(page, "Ravi Kumar", "ravi@gmail.com", "9876543210", "ABC Engineering College")  # event left out
    assert text_of(page, "error") == "Please select an event."  # Expected error message


def test_tc08_valid_registration(page):                     # TC08: successful registration
    before = int(page.find_element(By.ID, "count").text)    # Remember the participant count before registering
    fill_form(page, "Ravi Kumar", f"ravi{UNIQUE}@gmail.com", "9876543210",
              "ABC Engineering College", "Hackathon")       # Submit valid data with a unique email
    assert text_of(page, "success") == "Thank you Ravi Kumar! You are registered for Hackathon."  # Success message
    WebDriverWait(page, 10).until(
        lambda d: int(d.find_element(By.ID, "count").text) == before + 1)  # Wait until the count goes up by 1
    last_row = page.find_elements(By.CSS_SELECTOR, "#regTable tbody tr")[-1]  # Take the last row of the table
    assert "Ravi Kumar" in last_row.text and "Hackathon" in last_row.text  # It must show the new participant


def test_tc09_duplicate_email_rejected(page):               # TC09: same email cannot register twice
    fill_form(page, "Ravi Kumar", f"ravi{UNIQUE}@gmail.com", "9876543210",
              "ABC Engineering College", "Hackathon")       # Submit the email used in TC08 again
    assert text_of(page, "error") == "This email is already registered."  # Server returns 409 with this message


def test_tc10_clear_button(page):                           # TC10: Clear button empties the form
    page.find_element(By.ID, "name").send_keys("Test User")  # Type something in the name box
    page.find_element(By.CSS_SELECTOR, "button.reset").click()  # Click the Clear (reset) button
    assert page.find_element(By.ID, "name").get_attribute("value") == ""  # Name box must now be empty
```

Procedure:

1. Create the `event-app` folder with the application files above. Move into it, then build the image and start the container (the `.` in `docker build` means "this folder", so the command must run inside `event-app`):

   ```
   cd event-app
   docker build -t event-app:1.0 .
   docker run -d -p 3000:3000 --name event-app event-app:1.0
   docker ps
   ```

   Open `http://localhost:3000` in the browser and confirm the form loads. If `docker run` reports that the name `event-app` is already in use, remove the old container first with `docker rm -f event-app`.
2. Create the test project in a separate folder, with its own virtual environment, and install Selenium and the test tools:

   ```
   cd ..
   mkdir selenium-lab
   cd selenium-lab
   python3 -m venv .venv
   source .venv/bin/activate          # Windows: venv\Scripts\activate
   pip install selenium pytest pytest-html
   ```
3. Create `test_event_app.py` inside `selenium-lab` with the program above.
4. With the container still running, run the tests: `pytest test_event_app.py -v --html=report.html --self-contained-html`.
5. Open `report.html` in Chrome to view the test report.
6. Optional – if the app is also deployed on Minikube behind a NodePort Service named `event-app-service`, test it with the same script: `BASE_URL=$(minikube service event-app-service --url) pytest test_event_app.py -v` (Windows PowerShell: `$env:BASE_URL="http://<minikube-ip>:30080"; pytest test_event_app.py -v`).
7. Stop and remove the container: `docker stop event-app` and `docker rm event-app`.

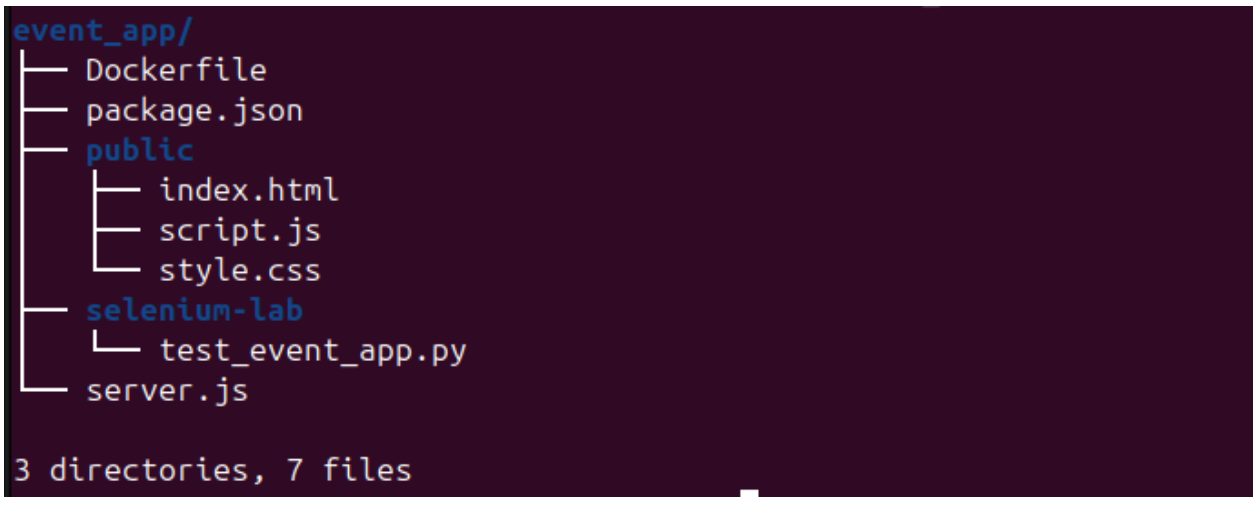
Troubleshooting – browser shows "Cannot GET /": the server is running but `public/index.html` is not inside the image. Check what the container holds:

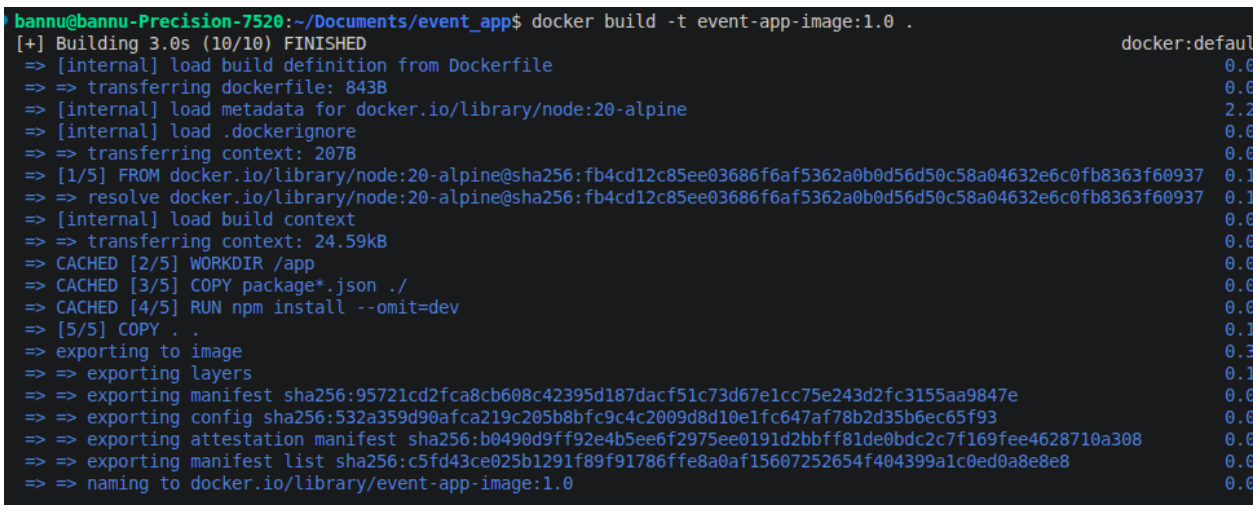
```
docker exec event-app ls -R /app/public
```

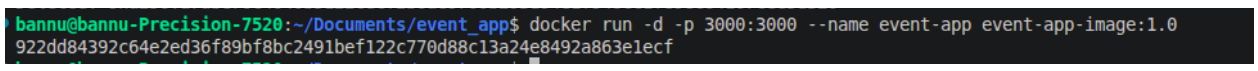
It must list `index.html`, `script.js` and `style.css`. Common causes: the three web files were saved directly in `event-app` instead of `event-app/public`; the folder is named `Public` (Linux inside the container is case-sensitive); or Windows saved the file as `index.html.txt` (turn on **View → File name extensions** to check). Fix the files, then rebuild and restart, because a running container does not see later file changes:

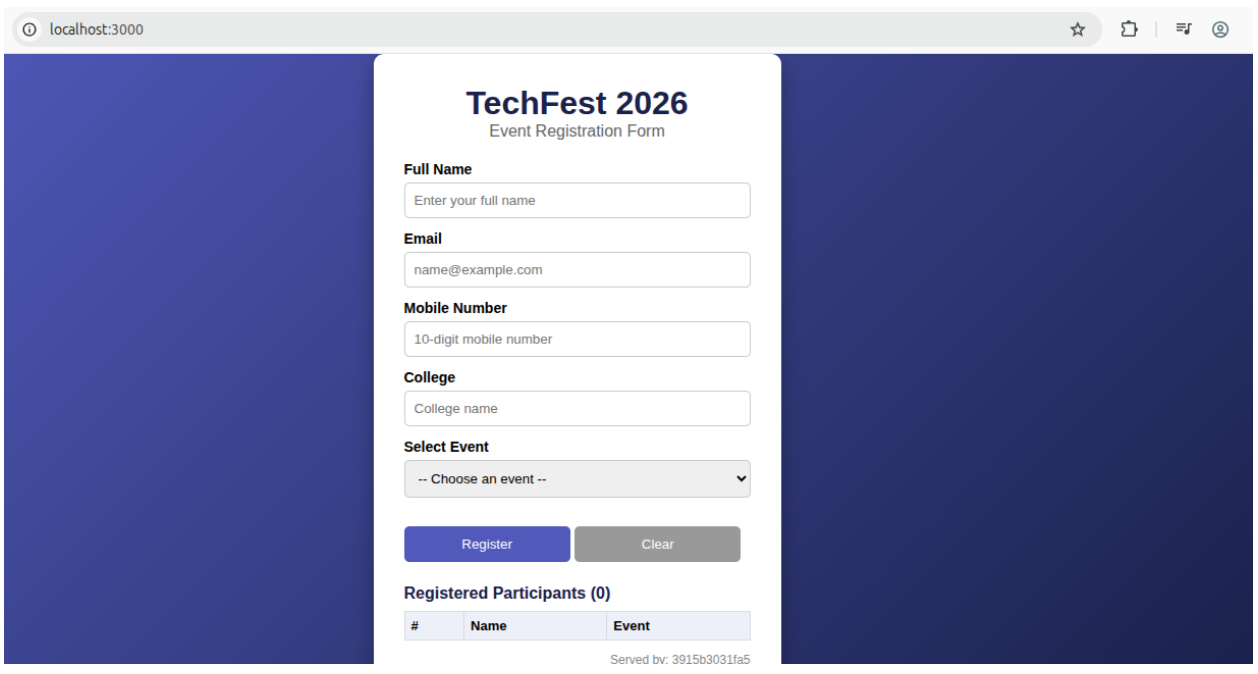
```
docker rm -f event-app
docker build -t event-app:1.0 .
docker run -d -p 3000:3000 --name event-app event-app:1.0
```

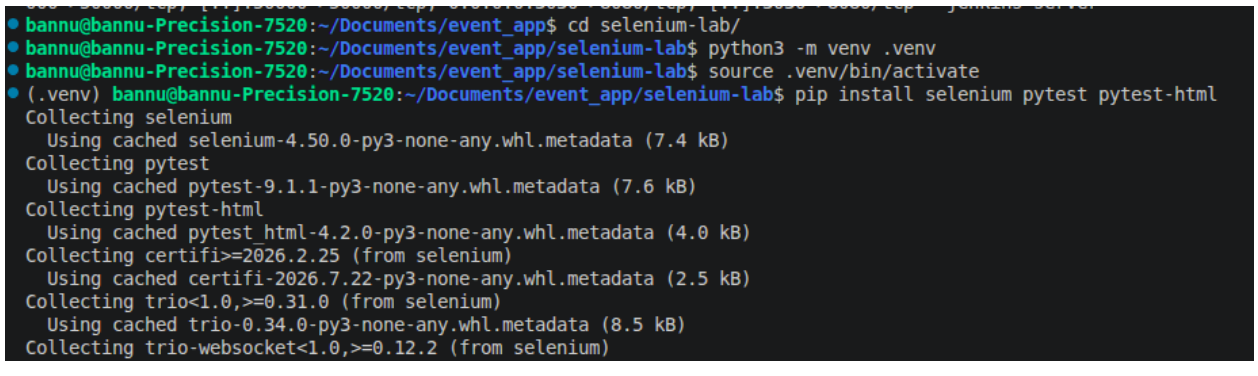
Output :











```
(.venv) bannu@bannu-Precision-7520:~/Documents/event_app/selenium-lab$ pytest test_event_app.py -v --html=report.html --self-contained-html
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/bannu/Documents/event_app/selenium-lab/.venv/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.0-34-generic-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'metadata': '3.1.1', 'html': '4.2.0'}}
rootdir: /home/bannu/Documents/event_app/selenium-lab
plugins: metadata-3.1.1, html-4.2.0
collected 10 items

test_event_app.py::test_tc01_page_loads PASSED                                                        [ 10%]
test_event_app.py::test_tc02_form_fields_present PASSED                                               [ 20%]
test_event_app.py::test_tc03_health_endpoint PASSED                                                   [ 30%]
test_event_app.py::test_tc04_empty_form_rejected PASSED                                               [ 40%]
test_event_app.py::test_tc05_invalid_email PASSED                                                     [ 50%]
test_event_app.py::test_tc06_invalid_phone PASSED                                                     [ 60%]
test_event_app.py::test_tc07_event_not_selected PASSED                                                [ 70%]
test_event_app.py::test_tc08_valid_registration PASSED                                                [ 80%]
test_event_app.py::test_tc09_duplicate_email_rejected PASSED                                          [ 90%]
test_event_app.py::test_tc10_clear_button PASSED                                                      [100%]

---------------- Generated html report: file:///home/bannu/Documents/event_app/selenium-lab/report.html ----------------
============================================== 10 passed in 13.13s ==============================================
```

Result: Ten Selenium test cases were developed for the containerized event registration application, covering page load, form elements, the health API, input validation, successful registration, duplicate detection and the Clear button. All tests passed and an HTML test report was generated.